중간 레이어 토큰 주입과 순환 계산을 통한 대규모 언어모델 추론 가속

Intermediate-Layer Token Injection and Cyclic Computation
for Efficient Autoregressive Decoding

연구 분야: LLM 추론 가속 · 적응적 계산 · 자기회귀 생성 | 2026. 11. 02.

연구 요약

본 연구는 토큰을 확정한 중간 레이어에서 다음 토큰의 계산을 이어가고, 추가 계산이 필요한 토큰에만 더 깊은 레이어와 순환 계산을 할당하는 언어모델 추론 방식을 제안한다. 일반적인 자기회귀 모델은 새 토큰마다 첫 레이어부터 마지막 레이어까지 처리한다. 제안 방식은 토큰의 출력 위치와 다음 토큰의 입력 위치를 연결하여 토큰별 계산 경로를 조절한다. 마지막 레이어에서도 출력을 확정하지 못하면 이전 상태를 첫 레이어로 전달하여 계산을 계속한다.

별도 학습을 하지 않은 Qwen2.5-0.5B-Instruct의 24개 레이어를 관측한 예비 조사에서, 8,192개 평가 토큰 중 52.77%는 어느 레이어에서든 원문 정답과 일치하는 예측을 보였다. 이들 토큰에서 정답이 처음 나타난 평균 깊이는 21.30층이었다. 이 관측은 중간 예측을 조사할 출발점을 제공하지만, 중간 입력이나 순환 계산의 유효성을 입증하지는 않는다. 본 연구는 실제 경로를 통과하는 모델의 공동 학습과 정답을 모르는 추론 정책을 통해 그 가능성을 검증한다.

1. 연구 배경 및 목적

LLM의 생성 비용은 모델 크기뿐 아니라 토큰마다 반복하는 순차 계산에 의해 결정된다. KV cache를 사용하는 표준 추론도 새 입력 토큰을 모든 Transformer 레이어에 통과시킨다. 반면 토큰별 예측 난이도는 다를 수 있으므로, 모든 토큰에 같은 깊이의 계산을 적용할 필요가 있는지 검토할 필요가 있다.

본 연구의 목적은 중간 예측, 다음 토큰 입력, 순환 상태 전달을 하나의 생성 경로로 학습하고, 동일한 품질 기준에서 실제 생성 지연을 줄일 수 있는 조건을 밝히는 것이다. 주된 연구 질문은 '이전 출력 레이어에서 다음 토큰을 처리하는 방식이 매 토큰을 첫 레이어부터 처리하는 방식보다 효율적인가'이다. 계산량 감소와 실제 속도 향상을 각각 검증한다.

2. 예비 조사: 최초 정답 레이어 관측

2.1 조사 방법

추가 학습 없이 사전학습된 Qwen2.5-0.5B-Instruct를 사용하였다. 원문의 prefix를 표준 인과적 attention으로 처리하고, 각 Transformer block의 출력을 기존 final normalization과 공유 LM Head에 연결하여 다음 토큰을 예측하였다. 원본 가중치는 변경하지 않았으며, 중간 예측은 관측에만 사용하였다.

표 1. 무학습 관측 실험의 설정

항목	설정
모델	Qwen2.5-0.5B-Instruct, Transformer 24층
데이터	WikiText-2 raw, 공식 validation의 5개 문서
평가 규모	8,192개 다음-token 예측 위치, 입력 chunk 21개
입력 구성	최대 512 tokens, overlap/context 64 tokens
추가 학습	수행하지 않음
정답 및 평가	원문의 다음 token ID exact match, teacher forcing

앞 문맥은 각 위치에서 원문 토큰으로 제공하였다. 모델의 앞선 오답을 다음 위치의 입력으로 누적하지 않았으며, 현재 정답과 미래 토큰은 해당 예측의 입력에 포함하지 않았다. 평가 단위는 단어나 문장 수가 아닌 tokenizer의 예측 위치다. 겹치는 입력 문맥의 정답 위치는 중복 집계하지 않았다.

표 2. 무학습 관측 결과

지표	관측값
어느 레이어에서든 정답이 나타난 위치	4,323 / 8,192개 (52.77%)
정답이 나타난 위치의 평균 최초 정답 깊이	21.30 / 24층
동일 집합의 최초 정답 깊이 중앙값	22층
24층까지 정답이 나타나지 않은 위치	3,869 / 8,192개 (47.23%)
미도달 위치의 비용을 24층으로 둔 전체 평균	22.58층

2.2 결과의 해석

21.30층은 모든 토큰에 대한 평균이 아니라 정답이 한 번 이상 나타난 위치의 조건부 평균이다. 또한 정답이 일찍 한 번 나타났다는 사실만으로 그 시점에 올바르게 출력을 확정할 수 있는 것은 아니다. 본 관측은 실제 레이어 생략, 중간 토큰 주입 또는 순환 생성의 성공을 보여주지 않는다. 평가 문서 수도 5개로 제한되어 있으므로, 제안 구조의 효용은 새로운 실험을 통해 독립적으로 검증한다.

3. 연구 가설 및 관련 연구와의 관계

3.1 연구 가설

가설 1 — 경로 학습 가능성. 중간 레이어 토큰 입력과 마지막→첫 레이어 상태 전달을 실제로 경험하도록 Transformer 본체와 전환 모듈을 공동 학습하면, 해당 경로에서도 다음 토큰 예측을 수행할 수 있다. 중간 표현에서 정답을 관측한 것과 별개로, 새로운 입력 경로 자체의 학습 가능성을 검증한다.

가설 2 — 다음 토큰 입력 위치의 효용. 동일한 데이터와 학습 예산에서, 이전 토큰을 확정한 레이어에서 다음 토큰을 시작하는 방식이 매 토큰을 첫 레이어에서 시작하는 방식보다 품질 대비 실제 block 방문 수를 줄일 수 있다. 입력 위치 변경으로 문맥 정보가 손실되어 품질이 악화되면 이 가설을 지지하지 않는 것으로 판단한다.

가설 3 — 선택적 순환의 효용. 추가 계산이 필요한 토큰에 한해 순환을 허용하면, 동일한 계산 예산의 비순환 대조군보다 예측 품질이나 해결 범위가 개선될 수 있다. 반복이 상태 정체만 일으키거나 같은 비용에서 개선이 없으면 순환의 이점이 없는 것으로 판단한다.

가설 4 — 실제 추론 가속. 정답을 모르는 출력 판정기와 LM Head, cache 유지, 순환 및 fallback 비용을 포함하더라도, 사전에 정한 품질 제약 안에서 표준 자기회귀 추론보다 생성 시간이 짧아질 수 있다. 이 가설이 본 연구의 최종 검증 대상이다.

3.2 관련 연구와 차별화할 질문

CALM은 생성 중 신뢰도에 따라 계산 깊이를 조절하며, 조기 출력에 따른 오차 전파와 누락된 과거 표현을 다룬다[1]. LayerSkip은 중간 출력 학습과 self-speculative decoding을 결합한다[2]. Mixture-of-Recursions는 토큰별 재귀 깊이 선택과 선택적인 KV 활용을 연구한다[3]. 따라서 중간 출력이나 반복 깊이 계산 자체를 새로운 기여로 주장하지 않는다.

본 연구가 구체적으로 검증할 대상은 이전 토큰의 출력 깊이와 다음 토큰의 시작 깊이를 연결하는 전환 규칙이다. 이 규칙에 상태 전달과 순환을 결합했을 때 얻는 이점과 비용을 분리한다. 관련 방법과의 비교에서는 모델, 학습량 및 구현 조건의 차이를 명시하고, 검증되지 않은 최초성이나 가속 배율을 주장하지 않는다.

4. 연구 방법론

4.1 중간 토큰 주입 및 순환 계산

각 Transformer block 뒤에 공유 LM Head를 사용하는 예측 지점을 둔다. 현재 레이어에서 다음 토큰이 확정되면 그 토큰의 embedding과 이전 hidden state를 결합하여 같은 레이어에 입력한다. 확정하지 못하면 다음 레이어로 진행한다. 마지막 레이어 이후에도 추가 계산을 허용할 때에는 현재 hidden state를 첫 레이어의 입력 공간에 맞게 변환하여 전달한다.

예를 들어 layer 7에서 A를 확정하면 A를 layer 7에 넣어 B를 예측한다. B가 layer 9에서 확정되면 다음 계산은 layer 9에서 시작한다. 이후 토큰을 layer 24까지 확정하지 못하면 상태를 유지하여 layer 1로 순환한다. 같은 token을 같은 초기 embedding에서 단순 재실행하는 동작과 구분한다.

4.2 실제 경로의 공동 학습

사전학습 가중치에서 시작하여 0.5B 모델의 Transformer 본체와 토큰 입력·상태 전달·순환 모듈을 함께 미세조정하는 것을 주 실험으로 계획한다. 원본 모델을 고정한 출력 해석만으로 가설을 검증하지 않는다. 학습 대상과 optimizer에 포함한 parameter를 기록하며, 이러한 공동 학습의 구현과 성능 검증은 본 연구에서 새로 수행할 항목이다.

입력이 A이면 정답은 다음 token B다. 실제로 방문한 각 예측 지점에서 B의 cross-entropy loss를 계산한다. 중간에 B를 맞히면 B를 같은 레이어에 넣어 다음 token을 학습한다. Episode 전체의 hidden state와 KV 계산 graph를 유지하여, 뒤쪽 token의 loss가 앞쪽 상태와 전환 모듈에도 전달되도록 한다. 한 episode 중간에는 optimizer를 변경하지 않는다.

학습 계산량은 episode 전체의 누적 iteration으로 제한한다. 첫 순회를 1로 두고 현재 target index가 i일 때 i+1까지 허용한다. 다음 순회가 이 한도를 넘으면 현재 loss를 남기고 정답 token을 입력으로 공급하여 layer 1에서 다음 target 학습을 계속한다. 이 강제 공급은 모든 학습 label에 도달하기 위한 정책이며, 직접 예측 성공과 별도로 집계한다.

4.3 문맥 접근과 실제 추론

초기 prompt는 full-depth prefill로 처리한다. 이후 layer별로 실제 방문한 token 위치의 K/V를 저장하고, 같은 위치를 재방문하면 K/V를 교체한다. 순환만으로 token 위치를 증가시키지 않는다. 중간 시작으로 누락되는 과거 K/V는 상태 전달만 사용하는 조건과 실제 보완 계산 조건으로 나누어 평가하고, 보완 비용도 포함한다.

경로 분석에서는 정답을 이용한 oracle 분기를 사용한다. 실제 추론에서는 신뢰도와 hidden feature를 이용한 판정기가 확정 여부를 결정하며, 정답을 분기에 사용하지 않는다. 판정기는 학습 split의 성공 여부로 학습하고 임계값은 validation에서 고정한다. 계산 budget 소진 시 원본 경로로 fallback하는 경우에는 prefix와 canonical cache를 복구하는 비용을 포함한다. 오답을 항상 탐지하거나 원본 출력 분포를 정확히 보존한다고 가정하지 않는다.

5. 데이터 및 학습 실험 설정

5.1 데이터 구성

주 데이터는 영어 Wikipedia 문서로 구성된 WikiText-2 raw이다[5]. 공식 train은 학습, validation은 모델 및 판정 기준 선택, test는 평가에 사용한다. 아래 전체 토큰 수는 실험과 동일한 Qwen tokenizer와 문서 복원 방식으로 집계한 값이다. 문맥 토큰과 loss의 정답 위치 수는 구분한다.

표 3. 전체 데이터와 기본 학습·평가 표본

분할	전체 토큰	표본 수	정답 토큰
Train	2,517,232	131,072	1,048,576
Validation	262,210	512	4,096
Test	298,938	1,024	8,192

기본 표본 하나는 연속된 문맥 32 tokens와 예측 대상 8 tokens로 구성한다. 문서를 순환 선택하고 random offset을 추출하되, 같은 split 안에서 정답 위치가 겹치지 않게 한다. 입력 문맥의 겹침은 허용한다. 3 epoch는 같은 1,048,576개 정답 위치를 세 번 학습하는 것으로, 서로 다른 약 314만 개의 정답을 의미하지 않는다.

짧은 생성에만 맞춰진 결과를 피하기 위해 예측 길이 32·128 tokens와 문맥 길이 128·512 tokens를 추가한다. 조건별 표본 수와 파일 hash를 고정한다. 일반화 평가는 긴 문맥의 마지막 단어를 예측하는 LAMBADA에서 수행하며, 단어 exact match와 tokenizer의 subtoken 지표를 구분한다[4].

5.2 주 학습 및 비교 조건

표 4. 제안하는 주 학습 설정

항목	계획
주 모델	Qwen2.5-0.5B-Instruct, 사전학습 가중치에서 시작
학습 범위	Transformer 본체와 입력·상태 전달·순환 모듈 공동 학습
학습량	기본 train 표본 3 epoch, 데이터 동일 조건 비교
Optimizer	AdamW, 초기 LR 1e-5, validation에서 LR 선택
Gradient	Episode 전체 역전파, clip norm 1.0
장치 및 반복	물리 GPU 1·2·3, 학습 seed 3개

학습률은 예비 후보 1e-5·3e-5를 validation에서 비교한다. 데이터 표본은 고정하고 학습 초기화·순서의 효과를 구분한다. 7B는 0.5B에서 유효한 조건을 선정한 뒤 자원 범위 안에서 확장하며, 학습 범위가 달라지면 그 차이를 보고한다. 본 표는 앞으로 수행할 공동 학습의 계획이며 완료한 학습 결과가 아니다.

6. 실험 설계

6.1 단계별 실험과 대조군

첫째, 무학습 layer별 관측을 더 많은 문서에서 재현하여 최초 정답 깊이와 미도달 비율을 확인한다. 둘째, 실제 주입·순환 경로의 공동 학습 전후를 비교한다. 셋째, oracle을 사용하지 않는 판정기를 적용하여 품질과 실제 추론 시간을 측정한다. 예비 관측과 경로 학습의 성과를 서로 다른 단계의 결과로 보고한다.

표 5. 주요 비교 조건

조건	다음 토큰 시작	순환	비교 목적
원본 표준 AR	Layer 1	없음	원본 품질·지연 기준
학습된 조기 출력	Layer 1	없음	중간 출력의 효과
학습 전 제안 경로	직전 확정 layer	허용	경로 변경 자체의 영향
공동 학습된 중간 입력	직전 확정 layer	없음	입력 위치 변경의 효과
공동 학습된 전체 경로	직전 확정 layer	허용	순환의 추가 효과

학습된 조건끼리는 학습 데이터, 학습량, optimizer 및 학습 parameter 범위를 가능한 한 맞춘다. 같은 조기 출력 기준과 fallback 정책에서 시작 위치 또는 순환 여부만 바꿔 비교한다. Oracle 경로와 실제 추론은 별도의 표로 보고한다.

6.2 구성 요소 제거 및 예산 실험

새 토큰 embedding만 넣는 조건, 이전 hidden state를 추가하는 조건, 학습된 입력 변환을 사용하는 조건을 비교한다. 순환은 사용하지 않는 조건, 단순 정규화 전달, 학습된 상태 변환으로 구분한다. 누락 KV의 보완 여부도 독립적으로 비교한다.

추론 예산은 모델 깊이의 1·2·4·8배에 해당하는 실제 block 방문 수로 설정한다. LM Head 확인 간격은 매 layer와 2·4개 layer 간격을 비교한다. 모든 위치에 출력 경로를 두는 것과 매번 vocabulary projection을 실행하는 것은 비용이 다르므로 호출 수를 함께 기록한다. 시간 제한 없는 loop 관측은 상태 정체 분석에만 사용한다.

6.3 통제 및 재현성

학습 전후 비교는 같은 target, prefix, 표본 목록을 사용한다. 신뢰구간은 토큰을 독립으로 간주하지 않고 문서 단위 paired bootstrap으로 산출한다. 모델·데이터 revision, 학습 범위, seed, 실제 처리 label 수, 방문 loss 수 및 GPU-hour를 보존한다. 기존 탐색 실험에서 관측한 test 표본의 노출 이력을 밝히고, 최종 설정은 validation에서 확정한 뒤 미사용 평가 표본으로 확인한다.

7. 평가 지표 및 성공 기준

7.1 예측 깊이와 경로 비용

표 6. 깊이 및 경로 평가 지표

지표	정의와 보고 원칙
최초 정답 깊이	정답이 나타난 토큰의 평균·중앙값·분위수와 전체 coverage를 함께 제시
공통 성공 깊이	학습 전후 모두 정답이 나타난 같은 target 집합에서 비교
실제 block 방문 수	순환과 재방문을 모두 누적하고 실패 비용 포함
직접 해결·정답 완주	Oracle에서 강제 공급 없이 맞힌 비율 및 episode 완주율
Loop·상태 정체	실제 마지막→첫 레이어 전환 수와 반복 상태, 미완주 비율
시스템 비용	LM Head 호출, KV 보완, fallback, peak memory를 포함

학습에서 정답을 강제로 공급하여 처리한 token은 직접 해결 수에 포함하지 않는다. 시간 제한으로 멈춘 loop는 우측 검열로 기록하고, 관측 loop 수는 필요한 수의 하한으로 해석한다. 완주 표본이 없으면 평균 필요 loop 수는 미확인으로 둔다. 실제 생성에서 출력 길이를 채운 것과 원문을 모두 맞힌 것은 구분한다.

7.2 생성 품질과 실제 속도

품질은 동일 reference prefix에서의 next-token NLL·perplexity, 원본 greedy 출력과의 token·sequence 일치도, LAMBADA 단어 정확도, 반복 생성 및 조기 EOS 비율로 평가한다. 원문 다음 token은 가능한 continuation 중 하나이므로 exact match만으로 자연어 품질을 판단하지 않는다. NLL은 정답을 분기에 사용하지 않는 고정 scoring 정책으로 측정한다.

속도는 같은 물리 GPU 한 장에서 같은 모델 크기, dtype, attention backend, prompt 및 생성 길이로 비교한다. Warm-up 후 CUDA 동기화와 wall-clock 측정으로 prefill, 첫 토큰 지연, token당 지연, 전체 지연, tokens/s 및 p50·p95를 기록한다. Batch size 1을 주 조건으로 두고 4·8을 확장한다. Trace 기록 비용은 속도 측정에서 제외하되 실제 분기와 cache·fallback 비용은 포함한다.

연구 목표는 원본 AR 대비 perplexity 상대 증가 5% 이내와 LAMBADA 정확도 감소 1%p 이내인 조건에서 전체 추론 시간 기준 1.2배 이상의 가속을 확인하는 것이다. 이는 달성한 수치가 아닌 사전 목표다. 계산 깊이가 줄더라도 실제 시간이 증가하면 가속 성공으로 판단하지 않으며, 품질–속도 곡선과 실패 조건을 함께 제시한다.

8. 추진 일정 및 기대 기여

표 7. 단계별 추진 계획

기간	주요 내용	산출물 및 판단 기준
1주차	무학습 관측 확대, 경로 및 공동 학습 구현	데이터 고정, 상태·gradient 전달 검증
2–3주차	0.5B 학습 전후 및 대조군 비교	동일 비용에서 품질·해결 범위 비교
4주차	입력·memory·순환·KV 제거 실험	상태 전환의 유효 조건과 병목 식별
5주차	정답을 모르는 판정기와 속도 측정	품질 제약 안의 실제 지연 비교
6주차	긴 생성, 외부 데이터 및 규모 확장	일반화 분석과 최종 보고

본 연구는 토큰을 확정한 깊이에서 다음 토큰을 이어 처리하는 계산 규칙을 명시하고, 그 경로에 적합하도록 모델을 학습하는 방법을 검증한다. 중간 출력의 조기성, 다음 입력 상태의 적합성, 순환의 효용을 분리함으로써 어떤 구성 요소가 품질과 계산 비용을 결정하는지 분석한다.

예상 기여는 중간 출력과 다음 입력 위치를 연결한 생성 경로, 실제 경로 공동 학습 및 cache 유지 방법, 그리고 oracle 가능성과 배포 가능한 가속을 구분하는 평가 체계다. 순환이 도움이 되지 않거나 시스템 비용이 절감 효과를 상쇄하는 경우에도 그 조건을 재현 가능한 결과로 보고한다. 현재 완료된 예비 근거는 2절의 무학습 layer 관측이며, 제안 구조의 공동 학습과 가속은 후속 검증 대상이다.

참고문헌

[1] Schuster, T., et al. (2022). Confident Adaptive Language Modeling. NeurIPS 2022.
https://arxiv.org/abs/2207.07061

[2] Elhoushi, M., et al. (2024). LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. ACL 2024.
https://arxiv.org/abs/2404.16710

[3] Bae, S., et al. (2025). Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation.
https://arxiv.org/abs/2507.10524

[4] Paperno, D., et al. (2016). The LAMBADA dataset: Word prediction requiring a broad discourse context. ACL 2016.
https://arxiv.org/abs/1606.06031

[5] Salesforce. WikiText dataset card. 사용 구성: wikitext-2-raw-v1.
https://huggingface.co/datasets/Salesforce/wikitext